

A Controlled Experiment on the Impact of the Object Constraint Language in UML-based Maintenance

L.C. Briand, Y. Labiche, H.-D. Yan
Software Quality Engineering Laboratory
Systems and Computer Engineering Department
Carleton University, 1125 Colonel By Drive
Ottawa, ON K1S5B6, Canada
{briand, labiche}@sce.carleton.ca

M. Di Penta
Research Centre on Software Technology
University of Sannio - Department of Engineering
Piazza Roma, I-82100 Benevento, Italy
dipenta@unisannio.it

Abstract

The Object Constraint Language (OCL) was introduced as part of the Unified Modeling Language (UML). Its main purpose is to make UML models more precise by providing a constraint language. For example, operation contracts and statechart guard conditions can be precisely defined using OCL. There has been an ongoing debate on the usefulness of using OCL in UML-based development, questioning whether the additional effort and formality were worth the benefit. This paper presents the results of a controlled experiment that investigates the impact of using OCL on model comprehension and maintainability. Current results show that, once past an initial learning curve, significant benefits can be obtained by using OCL in combination with UML diagrams.

1. Introduction

The Object Constraint Language [18] was proposed as a way to bring additional precision to analysis or design models defined with the Unified Modeling Language (UML) [2]. OCL is currently part of the UML standard and a new version (OCL 2.0) is about to become a standard along with UML 2.0. However, a number of authors recommend against using OCL [12]¹ or leave it out of their proposed methodologies [9]. Others recommend it only during low-level design [5]. Furthermore, very few organizations using UML currently make any use of OCL. Though it is clear that OCL brings precision to modeling activities and offers a number of potential benefits [3, 6, 11], the question comes down to assessing whether the additional effort and formality bring any tangible benefits in practice. This is akin to the old, on-going debate in software engineering regarding the degree of formality required in early phases of development to develop high-quality software. This paper reports on a controlled experiment performed in a university setting. The students involved are 4th year computer and software engineering students who have been carefully trained in UML-based software

development over several courses. As further discussed below, the choice of performing a controlled experiment was made to control for other extraneous factors which could have affected the results (e.g., Ability) and to ensure people were all adequately trained.

Though effort was a constant here—all students spent roughly the same time to perform the tasks—our goal was to determine whether OCL would make a practically significant difference. Results showed that the use of OCL, combined with UML, offer significant benefits, both in terms of model comprehension and maintainability, but only after a certain learning curve is overcome. This paper only reports on part of the experiment, additional results are available in [4].

The paper is structured as follows. Section 2 describes the experiment objectives and design. Section 3 reports on the results of each step of the experiment, and we draw conclusions in terms of practical significance and future work in the last section.

2. Experiment Planning

The structure of this section follows the template provided in [19], a well-known textbook on experimentation in software engineering.

2.1. Definition of the Experiment

The purpose of this experiment is to evaluate the impact of using OCL, combined with UML (class diagrams, statecharts, sequence diagrams), on understanding and modifying a UML analysis document [5, 13] and therefore its corresponding system logic. We are, however, not able to perform a cost-benefit analysis as effort in our experiment design is constant: Nearly all students spend the entire time available performing the tasks assigned to them. This is typical of controlled experiments where time is usually limited, and case studies in industrial settings will likely be required to assess the cost of using OCL.

Analysis documents are the first UML models to be produced in most methodologies. They are different from subsequent design documents as they do not provide architectural information but mostly focus on the functionality and logic of the system to be developed,

¹ “Unless there is a compelling practical reason to require people to learn and use the OCL, keep things simple and use natural language.” [12]

though they also yield an emerging class structure which is the starting point of the design phase. (These analysis models are incomplete as only application domain classes are documented.) Analysis is considered a critical stage of development as all subsequent phases depend on it.

2.2. Context and Material

The context of the experiment is a 4th year computer and software engineering course at Carleton University. Students have been carefully trained in object-oriented (OO) development with UML in at least three previous courses, with an increasing focus on software modeling. The focus of this last course (about 40 hours) is to make them use what was learnt in the context of a well-defined development process, including analysis and design phases. Two analysis documents are used for a Cab Distribution system (CD) and a Video Store system (VS), respectively. Those documents represent a complete Analysis including a set of consistent diagrams. For the versions containing OCL constraints, OCL was used to document the following aspects of the models, which are typically documented in natural language (see an example in Figure 1): operation preconditions and postconditions, class invariants, guard conditions in statecharts, path conditions in sequence diagrams. All these constraints provide more precise information on potentially important aspects of the analysis models which would otherwise be documented in natural language. A number of people consider, however, such additional effort to yield little benefit [12]. Both analysis documents (CD and VS) are approximately forty pages long and are somewhat comparable in terms of complexity and size, as illustrated in Table 1. We discuss below, in Section 2.5, how we prevented the differences from affecting our results.

Table 1. Description of Analysis Documents

Metrics	Systems	
	Cab Distribution	Video Store
# of Actors	7	8
# of Use Cases	10	12
# of Classes	20	18
# of associations	5	8
# of inheritance relationship	2	0
# of inheritance depths	1	0
# of Methods	55	66
# of Attributes	23	40
# of States	9	10
# of State Transitions	16	15
# Messages	48	39
# of OCL expressions	174	291

2.3. Hypothesis formulation

In this experiment, we investigate the impact of OCL on two dependent variables:

- *Comprehension (C)*: This variable captures the capability of subjects to answer questions about the system functionality and logic. This is measured as the number of correctly answered questions in a

carefully designed questionnaire, during a one hour and a half session.

- *Maintainability (M)*: This variable focuses on the capability of subjects to determine what elements of the UML analysis models will need to undergo change based on requirements change descriptions. This is measured, during a one hour session, based on a carefully designed questionnaire where a number of changes are described and subjects are expected to list all affected model elements (e.g., classes, operations, attributes).

Questionnaires and data collection procedures are further discussed in Section 2.6. One important point is that questions were specifically defined so that they could be answered with or without OCL constraints. It was, however, hypothesized that OCL constraints contained clearer and more precise information. For instance in Figure 1, OCL pre and post-conditions provide clearer, more precise information than the textual description (e.g., on the attributes and links modified) for operation `cancel()` in class `Reservation` (VS). States are further described with state invariants in OCL but are not shown here due to size constraints.

Figure 1. Example of operation description

<code>cancel()</code>
Description: Cancels the current reservation. The reservation object will transit to state <code>Cancelled</code> from states <code>Pending</code> or <code>Outstanding</code> (see description of state invariants)
Parameter-List:
Return-List:
Pre: <code>self.state=#Pending or self.state=#Outstanding</code>
Post: <code>self->whenCancelled=TimePoint.currentTime()</code> and <code>self.state=#Cancelled</code> and <code>self.copy->isEmpty</code> and if <code>self.copy@pre->notEmpty</code> then <code>self.copy@pre.state=#ForRent</code> endif

Based on the above variables, we can formulate the following null hypothesis (H_0) to be tested for each dependent variable: There is no difference in Comprehension and Maintainability between subjects working on UML Analysis documents including and excluding OCL constraints (denoted below as variable *Method*). The alternative hypothesis (H_a) is that OCL affects performance for our two dependent variables. To be more precise, H_a should be one-tailed: We expect OCL to *increase* performance.

In addition to whether or not OCL was used in the Analysis documents, we wanted to collect data regarding the ability of subjects with respect to UML modeling (denoted *Ability* below). This was considered important as it is a well-known fact that variations in human ability create substantial differences in experimental task outcomes. This may prevent us from seeing a significant effect of OCL on our dependent variables. We therefore measure Ability by using the grades of the students from a previous UML modeling course that we think is directly relevant to the tasks at hand. Two of the authors of this paper were instructors for this course. Ability is then used

to refine the data analysis and to increase our chances to see an effect associated with the use of OCL.

2.4. Selection of subjects

The subjects selected were the 38 students registered in our last, most advanced software engineering course as we wanted the subjects of our experiment to have achieved a reasonable level of technical maturity and knowledge with respect to UML-based, OO development. Students were familiar with the concepts of contracts (pre- and post-conditions), state invariants, and class invariants. We saw no need to screen students as variations in ability are also present in industrial settings and as most of these students will actually become certified engineers within a year, thus being representative of new graduates entering the software engineering profession. The experiment was part of a series of compulsory laboratory exercises. Students were told they would not be graded on performance but that they were expected to perform their tasks in a professional manner to obtain the points assigned to the lab. (All students were monitored by the authors during their lab work.) Students were aware that we were attempting to evaluate the impact of OCL as well as improving their practical training in UML and OCL but they were not aware of the exact hypotheses we were testing or what results we were hoping to obtain.

2.5. Experiment design and layout

We first specify the experiment layout: Students were assigned to one of 4 groups, and the tasks they performed over 4 laboratories (3 hours each), as well as the order of the tasks, are summarized in Table 2. The rows represent the four subsequent labs students went through (each lab being a week apart) and the three tasks they performed, whereas the columns are the 4 groups of subjects. The table cells show, for each group, which system they worked on and whether they had OCL constraints for it. Note that this paper presents only the results related to Laboratories 2 and 4. Results on whether OCL helps defect detection in UML models are presented in [4].

This experiment only involves one factor whose effect is of interest to us: whether or not OCL is used in Analysis documents. We could have used a simple, completely randomized one-factor design, but we wanted to account for individual differences in such a way as to increase the statistical power of our analysis and ensure this would not create any systematic bias in our results [8]. One way to do the latter is to adopt a randomized block design [19] whereas the former can be addressed by accounting for subjects' ability during data analysis. Another advantage of proceeding this way is that we can study the interaction between subjects' ability and OCL (see Section 2.7 on data analysis).

Students were therefore grouped (into "blocks" composed of 20 and 18 students, respectively) according to whether or not they had obtained a minimum grade of

B- in the previous course on UML and OCL. Admittedly, other ways to capture ability and form blocks could have been considered, but our focus here was on their UML modeling capability and, as instructor of their previous courses on this topic, we felt it was the best alternative. Each of the four groups was then randomly assigned students from both blocks in nearly identical proportions. It was also important for us, in order to facilitate the data analysis, that each of the four groups be of approximately the same size (~8 in this case). For each task, each subject worked individually on each of the two systems, using UML+OCL in one case and only UML in the other. Each student performed all the tasks that were planned regarding Comprehension and Maintainability on both systems. Students were also prevented from collaborating and this could easily be monitored as tasks were performed in a laboratory setting.

The rationale for making subjects work on both systems was to (1) maximize the number of data points (observations) so as to increase statistical power, (2) to ensure that the differences in systems' complexity would not bias the results (though we selected models of "similar" complexity), and (3) to give the opportunity to every student to learn the same material. However, when asking experimental subjects to perform several times similar tasks (e.g., impact analysis) and use the same techniques (e.g., OCL contracts), we are subject to learning or fatigue effects [8] and we need to ensure they do not confuse the results. This was done in two ways. First, two of the groups used OCL the first time they performed the task whereas the other two groups did so the second time around, thus ensuring learning effects would not be confounded with OCL effects. As described in Section 2.7, our data analysis procedure also accounts for learning effects and quantify them.

Second, we used 4 groups of participants instead of 2 in order to avoid ordering effects (e.g., learning or fatigue effects) being confounded with system effects. For example, if all participants in group 1 would have used first the VS system with OCL (and then the CD system without OCL) while all participants in group 2 would have started with non-OCL documents on the CD system (and then VS with OCL), we could have observed that OCL helps more on the VS system as people in group 2 had more time to further mature their knowledge of UML and OCL. We would have then been unable to assess learning effects or system effects as they would be entirely confounded.

Another result of our design choices is that all groups use each of the two systems twice in a row, sometimes using OCL first or second. One may therefore ask whether this is a threat to the validity of the results. First it is important to observe that when the system is used twice in a row, it is to perform a different task and the data collected will be part of separate analyses (i.e., for different dependent variables). So if there is a system

learning effect, say for Group 1 from Lab 1 to Lab 2, this will result in a better comprehension and maintainability performance. But that learning effect will affect *all* groups and we can still pool together the four groups' data for analysis purposes. On the other hand, the data of Lab 1 and Lab 2 will never be analyzed together. Furthermore, we expect that system learning effect to be weak as labs take place a week apart and students do not have access to documents between labs. Therefore, we also do not expect people to remember OCL expressions from one week to the other, and certainly not the level of detail where the performance of non-OCL groups would get closer to that of OCL groups. However unlikely, even in the situation where people would remember OCL expressions from one week to the other, this would result in decreasing the impact of OCL but could not create an inflated difference between OCL and non-OCL performance scores.

Table 2. Experiment Design and Operation

	Group 1	Group 2	Group 3	Group 4
Defect Detection (Lab 1)	CD (OCL)	CD (no-OCL)	VS (OCL)	VS (no-OCL)
Comprehension Maintainability (Lab 2)	CD (no-OCL)	CD (OCL)	VS (no-OCL)	VS (OCL)
Defect Detection (Lab 3)	VS (no-OCL)	VS (OCL)	CD (no-OCL)	CD (OCL)
Comprehension Maintainability (Lab 4)	VS (OCL)	VS (no-OCL)	CD (OCL)	CD (no-OCL)

Two additional, related issues that can be raised from Table 2 are the possible exchange of information among students between labs and the possibility they might have kept working on the tasks outside the laboratory environment, thus invalidating our assumption that the effort spent on the tasks is nearly constant. This was prevented in several ways. Students were not aware of the detailed plan for the labs and therefore did not know they would have to perform the same tasks as their fellow students from other groups at a later point. Second, as mentioned above, students did not have access to the lab material between labs, whether the system descriptions or questionnaires. As a result any collaboration or continuation of the work between labs was unlikely and difficult.

2.6. Instrumentation

Besides Analysis documents, a number of forms were used in the experiment. A pre-lab survey questionnaire was distributed to obtain information about the background of the experiment participants, e.g., their UML experience, the program they belong to, their industrial experience. After the completion of both Comprehension and Maintenance tasks, forms were also distributed that asked specific questions about the system. Comprehension questions ask the students about the system internal logic and functionality. Figure 2 shows an example of such question for the VS system. There were

20 such questions for each system, amounting to a maximum score of 20.

Figure 2. Example Comprehension question

Which attributes/associations should be checked to verify that a reservation is outstanding?

- The "whenOutstanding" is not empty. The "whenFulfilled" is empty. The "whenCancelled" is empty. The "whenExpired" is empty.
- The "whenOutstanding" is not empty. The "whenFulfilled" is empty. The "whenCancelled" is empty. The "whenExpired" is empty. The size of "heldcopy" is one.
- The "whenOutstanding" is not empty. The "whenFulfilled" is empty. The "whenCancelled" is empty. The "whenExpired" is empty. The "heldcopy" is empty.
- The "whenOutstanding" is not empty. The "whenFulfilled" is empty. The "whenCancelled" is empty. The "whenExpired" is empty. The size of "heldcopy" is >0.
- Other answer: _____

Maintenance questions are focused on impact analysis. Students are provided with change descriptions and are asked to list model elements that will have to undergo change (in the model or corresponding code). Figure 3 shows an example of such question for the VS system. There were 6 such questions for each system, amounting to a maximum score of 71 and 60 for Cab and Video, respectively (data were normalized), i.e., corresponding to the total number of model elements undergoing change.

Figure 3. Example maintenance question

The video store company decides to limit the maximum number of unfulfilled reservations that a Member can perform in a day. Each member may have a different limit.

- Which class(es) must be modified?
- Which modifications (addition, change of attributes, operations, and/or operations' contracts) must be made in this (these) class(es)?

All comprehension and maintenance questions were selected according to a number of criteria:

- They had to cover different aspects/parts of the systems, to the largest extent possible.
- They should not be trivial or nearly impossible to answer based on the available information.
- They could be answered with or without OCL. If we had introduced questions that could only be answered with OCL expressions then the experiment would have no point as the results would then become obvious. Such questions are, anyway, difficult to find, and are expected to be rare in practice.
- Last but not least, the questions had to be relevant, that is had to concern information of interest. In other words, we had to feel that those would be questions that software engineers could ask about the system. For example, maintenance changes had to be plausible changes.

In addition, because we wanted to gain enough insight to strengthen and explain our results, we distributed survey questionnaires after the execution of each Comprehension and Maintainability task. Questions asked students about whether they had enough time to perform

the task, how easy the task was to perform, whether they understood the system's functionalities, or whether the lab instructions were clear and easy to follow. Moreover, for those using OCL, questions were asked about the amount of time used to read OCL expressions, how easy they were to understand, and how helpful they were perceived to be. Most questions were answered on a Likert scale from 1 (Strongly agree) to 5 (Strongly disagree), following recommended templates [14]. Figure 4 shows an example questionnaire for the comprehension task.

Figure 4. A survey questionnaire example

You are required to rate your level of agreement of the following statements. There are 5 levels of agreement: 1 – Strongly agree 3 – Not certain 5 – Strongly disagree 2 – Agree 4 – Disagree Please mark the corresponding box dark to provide your level of agreement of each statement.		
1.	☐	It took me a lot of time to understand the system before I could answer any questions.
2.	☐	I did not have any problems to understand what the system is doing
3.	☐	I had no difficulties to complete all the questions in this module.
4.	☐	I hope there could have been more time for me to finish this module.
5.	☐	Some of the questions of this module were not easy to answer.
6.	☐	The lab instruction is clear and easy to follow.
7.	☐	The OCL expressions in the document were easy to understand.
8.	☐	The OCL expressions in the document made the system easier to understand.
9.	☐	The OCL expression in the document helped to clear up some ambiguities.
10.	☐	The OCL expression in the document helped me to answer the questions in this module.
11.	☐	How much time did you spend in understanding the OCL expressions during this lab (scale: <25%, ≥ 25% and <50%, ≥ 50% and < 75%, ≥ 75%)?

2.7. Analysis Procedure

We are only interested in the effect of one factor—the presence or absence of OCL expressions in an Analysis document—on our two dependent variables: Comprehension and Maintenance. A simple statistical test (e.g., t-test) [7, 10] would do for that purpose as our factor has only two levels. Because most of our students performed the same task with and without OCL, we can use a paired t-test² and gain additional statistical power by removing individual differences [7]. We also need to consider possible ordering effects which are impossible to avoid (e.g., learning effects) and determine whether it interferes with our analysis. To do so, we will test whether the difference between UML+OCL and UML-Only scores can be explained by whether OCL is used in Lab 2 or Lab 4. If this is the case, the data of each Lab may have to be analyzed independently.

Because we expect student ability to have a strong effect on the task results, and though we control for ability in our design, in order to refine our data analysis, we will use a two-way ANOVA using Ability as a factor as well. Though we controlled that effect and tried to

minimize it by using comparable system models, a three-way ANOVA that includes a system variable could be considered as well, but the available data does not permit it and this is left to future work. The reason why two-way ANOVA may be interesting is two-fold [7]. First, this allows us to investigate interaction effects between ability and the use of OCL (Method). Perhaps, OCL has a different effect on students of different ability. Second, by introducing ability into the analysis, the variation it explains is removed and the analysis of the effect of OCL becomes as a result more powerful, e.g., we are more likely to see any significant effect if there is one.

When looking at ANOVA result tables, we want to determine whether sample means in the groups formed by Ability and Method categories are significantly different, i.e., that the independent variables have a significant impact on the dependent variables (i.e., Comprehension, Maintainability) and that the variations in means could not have been obtained by chance alone. This is typically captured by the results of a F-test as a p-value [7]: the probability that we could obtain the results we observe if the populations from which the group samples are drawn had equal means (e.g., in terms of Comprehension score). Another standard statistic measures the strength of the relationship between each of the independent variables and the dependent variable. It is denoted as E^2 and computes the relative improvement in our ability to predict the value of the dependent variable when we know what factor level (value, category) an observation belongs to [10]—it is in many ways similar to R^2 in regression analysis. Examples of the above statistics are provided while discussing the first table in Section 3.

The ANOVA procedure is based on a number of assumptions. Each combination of factor levels (e.g., OCL and high Ability) has to be assigned subjects in a random fashion (with possibly blocking as a constraint). It should also have an equal number of observations (subjects). The dependent variable should be, for each factor combination, normally distributed and the variance should be equal. Obviously, the last two assumptions are nearly never met in practice. But ANOVA is known to be very robust to departures from these assumptions [10], especially when the sample sizes are equal or similar for each factor combination. If the differences are not too large, unequal sample sizes do not prevent the use of ANOVA, it just makes it more complex to interpret its results: The results may depend on the order in which factors are included in the model. Without further expanding on this, due to size constraints, in our experiment the differences in sample sizes exist but are rather small (see Tables in Section 3). As expected, departures from normality and equal variance assumptions exist, but they are only mild. A detailed analysis shows that differences in variance are not statistically significant and the dependent variable distributions do not show extreme outliers or multi-modal

² In the text below, we only report results for parametric tests. Equivalent non-parametric tests were tried and made no difference.

distributions, thus exhibiting only mild departures from normality. Moreover, we are in a situation where we are only interested in the effect of one variable (OCL) and the other factors are merely used to refine this analysis. In that case, it is recommended to include the important variable last in the model. In other words, we test to which extent OCL explains the remaining variation after the effect of Ability has been accounted for. Though the differences in sample sizes are very small in our case, we will follow this procedure to be on the safe side.

With respect to the survey questionnaires, one goal is to detect differences (e.g., in terms of their perception of the usefulness of OCL) between groups using UML+OCL and UML-Only. Such data are used to better explain and support our quantitative results. A simple t-test [7] for independent samples is used for that purpose. This test is one-tailed as we expect OCL to have an effect in a specific direction based on the ANOVA results (e.g., questions are easier to answer and therefore exhibit better scores with OCL). When comparing differences in the central tendencies of questionnaire data between subsequent labs our goal is to monitor for learning effects and explain differences in results that may be observed. Since groups of students are identical when comparing subsequent labs, we use a statistical test for paired samples, the paired t-test [7]. This test is also one-tailed as we expect a learning effect over subsequent labs and therefore a performance improvement in terms of questionnaire scores.

2.8. Threats to validity

The main threats to validity in this experiment are related to external validity. Like any student experiments, the issue of whether the subjects are representative of software professionals arises. Recall that—and it is unusual for student experiments—our 4th year engineering students are probably better trained at software modeling with the UML and OCL than most software professionals. So, if there is an issue here, it is not the lack of knowledge and experience of our subjects, but rather that they represent the ideal, unusual case of people having been through a thorough UML training. However, an increasing number of software engineering graduates with such modeling skills are now integrating the software industry and should therefore become, slowly but surely, the norm. Moreover, the difference between students and professionals is not always clear cut, as reported in a recent experiment [1]: the performance of undergraduate and graduate students (whose expertise is similar to the one of our students) at maintaining a UML design was not found to be very different from the performance of junior and intermediate professional consultants.

Another external validity issue, which is unfortunately inherent to controlled experiments, is the size and complexity of our system models. The question is then whether OCL would have an equal, weaker, or stronger

impact has the system models been vastly more complex. We cannot answer that question and we see controlled experiment such as the one presented here as a first step before assessing whether actual case studies in industry settings should be even considered.

In terms of internal validity, we have prevented any confounding effect due to the system being used, the ability of students and the order of performing the tasks.

Construct validity is related to our measurement instruments, including the comprehension and maintenance questionnaires. Questionnaires were produced by one of the authors, who was not involved in the modeling of the two systems, so as to cover as many parts of the models as possible, while being answerable within the time available. Questions also had to be answerable with or without OCL so as not to bias the results in favor of using OCL. This is, however, a conservative decision as questions that require OCL to be answered may actually exist. Questions were phrases so as to be answered either as multiple choices questions (Comprehension) or by providing a list of model elements (Maintenance), thus both leading to an easy measurement of the subject performance.

The survey questionnaires focus on capturing the perception of the subjects on their tasks. They are not meant to (statistically) show the impact of the independent variable (Method) on our dependent variables but, rather, our objective is to support and explain our quantitative results by providing qualitative insight from the questionnaire data. We followed standard ways of designing survey questionnaire [14] and phrase subjective questions so as to avoid bias and increase reliability of answers. However, a possible bias could have been the following: When asked about their perception of the usefulness of OCL, students could have answered what they thought was expected by the experimenters. Evidence shows this was, however, not the case, as discussed in Section 3.

3. Analysis Results

We first start this section with some simple preliminary analysis that will help justify our subsequent analysis steps. Sections 3.2 to 3.5 then present the results of our data analysis one lab at a time, using standard descriptive statistics, ANOVA tables, and graphs of means [7]. The main reason for looking at each lab separately is to account for the learning effects that are suggested by our preliminary analysis when answering our research question about the impact of OCL. In each subsection, *Descriptive statistics* tables show number of observations (*Size*) and mean values (*Mean*), i.e., mean number of correct answers during comprehension and maintenance, whereas *ANOVA tables* report ANOVA analysis results in a standard form. Different significant thresholds (α -level) are used across the literature and we consider significant (highlighted in tables) any p-value smaller than 0.05.

3.1. Preliminary Analysis

The simplest way to answer our research question is to perform a one-tailed, paired t-test based on all students' scores when using UML+OCL and UML-Only. Table 3 shows that, OCL has a statistically significant impact for Comprehension, and though there is a (small) difference in the expected direction in terms of Maintenance scores, it is not statistically significant.

Table 3. Paired t-test to compare UML+OCL and UML-Only

Dep. Variable	mean(OCL)	mean(no_OCL)	p-value (t)
Comprehension	0.493	0.429	0.028
Maintenance	0.368	0.317	0.159

In order to understand why this is the case, we analyzed whether or not learning effects could be responsible of blurring the effects of OCL³. We performed a one-tailed t-test (Table 4) to determine whether the difference between UML+OCL and UML-Only scores is smaller when OCL was first used in Lab 2 (i.e., OCL was used the first time the task was performed, denoted as O_N) than in Lab 4 (i.e., OCL was used the second time the task was performed, denoted as N_O). Table 4 shows the mean differences (i.e., those are differences in percentages of maximum scores) between N_O and O_N, as well as one-tailed p-values.

Table 4. Independent samples t-test to assess the impact of task ordering

Order of Method	mean(N_O)	mean(O_N)	p-value (t)
Comprehension	0.114	0.014	0.0613
Maintenance	0.138	-0.043	0.0334

Table 4 suggests that the order in which OCL is applied has a significant effect on the difference between UML+OCL and UML-Only performances⁴. In other words, O_N students benefit less from OCL than N_O students. To us, this suggests that in the O_N case, another effect cancels the OCL effect whereas in the N_O case, it perhaps inflates it. That effect can only be learning effects as all other plausible effects have been controlled for (i.e., system, ability). For this reason, in the subsequent analysis steps, we analyze the data of each lab in an independent fashion so that we can expect students to have achieved a similar level of training and experience.

3.2. Laboratory 2 – Comprehension (C)

Results in Lab 2 for Comprehension only show a weak effect (p-value = 0.08 and $E^2=0.097$) for Ability (Table 6). This is also visible in Table 5 and Figure 5. It is interesting to note that all means in Table 5, whether using OCL or not, are below 10. This means that less than 50% of the questions were answered correctly. As further discussed below, we interpret this data as a lack of

³ Indeed, we expect learning effects rather than fatigue effects as our labs are taking place a week apart.

⁴ The p-values are not very small but recall we analyze small samples here and only large differences would result into very low p-values.

training and familiarity with the task. Even trained, 4th year engineering students are not used to handle large analysis models and documents of this kind of complexity. The course in which this experiment takes place is their first opportunity to do so.

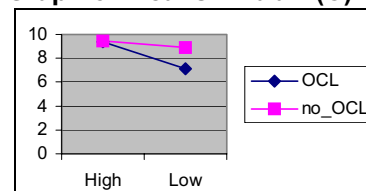
Table 5. Descriptive statistics – Lab 2 (C)

	Ability					
	High		Low		All	
Method	Mean	Size	Mean	Size	Mean	Size
OCL	9.38	8	7.14	7	8.33	15
No_OCL	9.44	9	8.88	8	9.18	17
All Methods	9.41	17	8.07	15	8.78	32

Table 6. ANOVA table – Lab 2 (C)

Source	DF	Sum of Squares	Mean Square	F Ratio	p-value
Ability	1	15.5741	15.5741	3.2585	0.0818
Method	1	6.4403	6.44032	1.3475	0.2555
Interaction of Ability, Method	1	5.4855	5.4855	1.1477	0.2932
Residual	28	133.829	4.77962		
Total	31	159.468			

Figure 5. Graph of means – Lab 2 (C)



3.3. Laboratory 4 – Comprehension (C)

In Lab 4, the Comprehension results are dramatically different from Lab 2 (Table 7 and Table 8). Both Ability and Method (OCL) have clearly a positive effect (p-value = 0.035 and 0.003, respectively) on people's capacity to answer questions about the system's logic, whether they have a low or high Ability (the interaction effect is not significant). The size effect of Method is $E^2=0.27$, which is practically significant and even higher than the effect of Ability ($E^2=0.12$). This result is further illustrated by Figure 6, which shows that the effect of OCL is similar whether students have a low or high Ability.

The difference between Lab 2 and Lab 4 for Comprehension can be explained by comparing their questionnaire analysis results. The analysis of the survey questionnaire for Lab 4 is consistent with the ANOVA results: Subjects using OCL spent less time understanding the system (p-value=0.007, mean(OCL)=3.87 vs. mean(non-OCL)=2.86) and found the comprehension questions less difficult to answer (p-value=0.00003, mean(OCL)=3.5 vs. mean(non-OCL)=1.92). Note that these results were not observed in the questionnaire data of Lab 2 where no question led to significantly different scores between the OCL and non-OCL groups.

Furthermore, the number of questions correctly answered is significantly higher for the OCL group, whether for low or high Ability students, in Lab 4 (10, 13)

than in Lab 2 (7.1, 9.4) (Figure 6): Using OCL increases, on average, by 46% and 43% the number of correct answers for high and low Ability students, respectively.

These results suggests that in Lab 4 people were better prepared to use and take advantage of OCL expressions and better understood how to analyze UML documents to answer questions. This can only be explained by the fact they had additional time to mature their understanding of OCL/UML and the task to be performed—It cannot be the result of a difference in skills or system, as these factors were controlled for in our design. In short, the differences between Lab 4 and Lab 2 are due to learning effects with respect to both the task and modeling skills, though we believe the former was far stronger.

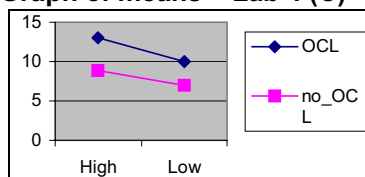
Table 7. Descriptive statistics – Lab 4 (C)

	Ability					
	High		Low		All	
Method	Mean	Size	Mean	Size	Mean	Size
OCL	13	7	10	7	11.5	14
No_OCL	8.86	7	7	7	7.93	14
All Methods	10.93	14	8.5	14	9.71	28

Table 8. ANOVA table – Lab 4 (C)

Source	DF	Sum of Squares	Mean Square	F Ratio	p-value
Ability	1	41.2857	41.2857	4.9828	0.0352
Method	1	89.2857	89.2857	10.7759	0.0031
Interaction of Ability, Method	1	2.2857	2.2857	0.2759	0.6042
Residual	24	198.857	8.2857		
Total	27	331.714			

Figure 6. Graph of means – Lab 4 (C)



3.4. Laboratory 2 – Maintenance (M)

Like for Comprehension, OCL does not have any effect in Lab 2 (Table 10, Figure 7). What is different though, is that Ability does not have any significant effect either. Results also show that on average, like Comprehension, only 30% of change impacts are correctly identified (Table 9). This is clearly another indication that, despite their training and many courses, they are not yet fully prepared for the task at hand, on documents of such scale and complexity. This may be a reason why we do not see any effect at all for Ability and Method.

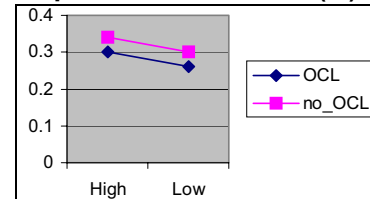
Table 9. Descriptive statistics – Lab 2 (M)

	Ability					
	High		Low		All	
Method	Mean	Size	Mean	Size	Mean	Size
OCL	0.3	8	0.26	7	0.28	15
No_OCL	0.34	9	0.3	8	0.32	17
All Methods	0.32	17	0.28	15	0.3	32

Table 10. ANOVA table – Lab 2 (M)

Source	DF	Sum of Squares	Mean Square	F Ratio	p-value
Ability	1	0.012460	0.012460	0.6091	0.4417
Method	1	0.014083	0.014083	0.6884	0.4137
Interaction of Ability, Method	1	0.000031	0.000031	0.0015	0.9691
Residual	28	0.572806	0.0204		
Total	31	0.599221			

Figure 7. Graph of means – Lab 2 (M)



3.5. Laboratory 4 – Maintenance (M)

As for Comprehension, the results for Lab 4 are very different from Lab 2. Results (Table 12) show that both Ability and Method (OCL) have a significant impact (p-value=0.006 and 0.029, respectively). The ANOVA results do not show a significant interaction effect though Figure 8 seems to indicate that the effect of OCL is stronger for low Ability students than for high ones (It increases, on average, by 122% and 13.3% the number of correct answers, respectively). The size effect of the impact of Ability and Method are $E^2=0.22$ and $E^2=0.13$, respectively. So we see that the impact of OCL is still practically significant as it represents 60% of the impact of Ability. As for comprehension, the differences in results between Lab 2 and Lab 4 are partly explained by the questionnaire analysis that was presented in Section 3.3⁵: Subjects in the OCL groups spent less time to understand the system. In addition, subjects also found the maintenance task easier to complete (p-value=0.006 with mean(lab2)=2.8 and mean(lab4)=2). Subjects in Lab 4 had less difficulty answering questions (p-value=0.011 with mean(lab2)=3.1 and mean(lab4)=2.55) and perceived OCL as more helpful (p-value=0.036 with mean(lab2)=3.23 and mean(lab4)=2.5)). Note that such results were not observed in the Lab 2 questionnaires.

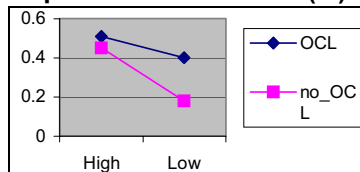
Table 11. Descriptive statistics – Lab 4 (M)

	Ability					
	High		Low		All	
Method	Mean	Size	Mean	Size	Mean	Size
OCL	0.51	8	0.4	7	0.46	15
No_OCL	0.45	7	0.18	7	0.31	14
All Methods	0.48	15	0.29	14	0.39	29

⁵ The survey questionnaires were in part the same for Comprehension and Maintenance tasks.

Table 12. ANOVA table – Lab 4 (M)

Source	DF	Sum of Squares	Mean Square	F Ratio	p-value
Ability	1	0.2552	0.2552	9.0546	0.006
Method	1	0.1504	0.1504	5.3357	0.029
Interaction of Ability, Method	1	0.0483	0.0483	1.7154	0.202
Residual	25	0.7047	0.0281		
Total	28	1.1597			

Figure 8. Graph of means – Lab 4 (M)

3.6. Discussion of results

Before focusing on the results per se, let us discuss the variation in number of participants across labs. As it is the case in many controlled experiments, not all subjects have participated to all labs and, as a result, for each dependent variable, we observe a loss in the number of subjects. However, the losses for Maintenance and Comprehension tasks are small (3 and 4 subjects were lost, respectively). This is, however, a potential threat to validity but, fortunately, the proportion of subjects across Method and Ability categories remain very similar (as shown in descriptive statistics tables) and there is therefore no reason to consider the results have been affected. This is also confirmed when looking more closely at the distributions of grades on which Ability is based.

One common trend across all laboratories and tasks was that there was a strong learning effect between the student's first attempt at a task and the second one. This was expected as, despite their intensive training in UML and OCL, performing inspections or answering questions about a 40 pages UML model (including OCL expressions) turned out to be very different from classroom exercises. In other words, students got trained in using OCL in larger, more complex UML models.

In Lab 4, once the initial learning curve completed, there is clearly a significant, positive effect of OCL on students' capability to answer questions about the system logic, and perform impact analysis. ANOVA results show the effect of OCL is simply additive to that of student ability, though for Maintenance results a larger sample might have provided a significant interaction effect. It further suggests that the effects of OCL are statistically and practically significant: It explains between 13% and 27% of the variance by itself (E^2). It increases the number of correctly answered comprehension questions by more than 40%, and the impact analysis completeness by 13% and 122%, for high and low ability students. These results are also supported by our survey data, bringing insight into the reasons for the impact of OCL.

The results of this study clearly show that OCL, as a constraint language complementary to UML, can be useful in helping people understand models and in facilitating the change of such models. However, the interaction of modeling ability with this effect needs to be further studied and understood (as suggested by the Maintenance results). Furthermore, gains in using OCL can only be observed if people have sufficient training and have a minimum experience in using real-scale UML models. That training was substantial for the students that took part in this experiment. Though they do not have the experience of many software professionals, it is our observation that their UML and OCL training was much more thorough than most software engineers in industry. Our results suggest that people will only benefit from using more precise modeling through the use of OCL once they receive a much more extensive training, in both UML and OCL, than what they typically have in the current state of practice.

4. Conclusions

This paper investigates an important methodological aspect of the use of the Unified Modeling Language (UML) [2] for OO Analysis and Design: Whether or not, the Object Constraint Language (OCL) [18], which is part of the UML standard, should be used to augment the precision of models. Though in the specific context of UML, this question is very much akin to the ongoing debate about the required level of formalism and precision in software development [16, 17].

In order to investigate this question we performed an experiment with 4th year software/computer engineering students who received a very thorough training in UML and OCL. This experiment looked at the impact of using OCL on two variables capturing the capability to: (1) answer questions about the underlying logic of the system and (2) perform an impact analysis of changes using the UML model. The experiment design was defined so as to prevent confounding effects from the widely varying ability of students to use models and the different characteristics of the two systems we use. It was important to ensure the internal validity of our controlled experiment by making sure the trends we observe are due to OCL and not another extraneous factor.

Results show that OCL has a positive impact on the two abovementioned variables. That impact is both statistically and practically significant. The fact that we see an effect is one thing but, more interestingly, that effect can be very large (e.g., up to a 122% increase for maintenance). This then suggests that the cost-benefit of using OCL is worth investigating further in industrial settings, especially considering that OCL is also potentially beneficial for other reasons not investigated here, e.g., its impact on the quality of the resulting implementation.

However, the above results were only observed the second time students performed the tasks. In other words, we observed a learning effect while running the experiment which was also confirmed by the data of the survey questionnaires we distributed after each lab. This was in fact to be expected as, though the students had an in-depth UML and OCL training, they did not have much experience with documents of the level of complexity exhibited in the UML models that were used in the experiment. They also had little experience with respect to the specifics of the tasks to be performed.

To conclude, the results of our experiment showed that OCL had the potential to significantly improve people's ability to inspect and modify UML models, but that this required significant training and experience. This later result is not so surprising as it is often the case that new methodologies or technologies take substantial time to assimilate before they start paying off [15]. From a practical perspective, this means that it is unlikely that people will benefit from using OCL in UML models unless they have proper training. Based on our personal observations, this required level of training is much more substantial than what is typically observed across the software industry. This is, however, likely to change over time as new software engineering graduates permeate software development organizations and as the UML standard gets a firmer penetration.

A replication of this experiment has been performed and data is currently being analyzed. Further experiments are planned as, in this paper, we only investigated a partial aspect of the impact of OCL on UML-based software development. Further studies are necessary to investigate, for instance, the impact of using OCL on the source code quality in subsequent development phases or how it supports testing activities [3].

Acknowledgements

This work was partly supported by a Canada Research Chair (CRC) grant. Lionel Briand and Yvan Labiche were further supported by NSERC operational grants. This work is part of a larger project on testing object-oriented systems with the UML (www.sce.carleton.ca/Squall). The authors would like to thank the students who participated in the experiment.

References

- [1] E. Arisholm and D. Sjøberg, "A Controlled Experiment with Professionals to Evaluate the Effect of a Delegated versus Centralized Control Style on the Maintainability of Object-Oriented Software," Simula Research Laboratory, Technical Report 2003-6, <http://www.simula.no/>, 2003.
- [2] G. Booch, J. Rumbaugh and I. Jacobson, *The Unified Modeling Language User Guide*, Addison Wesley, 1999.
- [3] L. C. Briand, Y. Labiche and H. Sun, "Investigating the Use of Analysis Contracts to Improve the Testability of Object-Oriented Code," *Software - Practice and Experience*, vol. 33 (7), pp. 637-672, 2003.
- [4] L. C. Briand, Y. Labiche, H.-D. Yan and M. Di Penta, "A Controlled Experiment on the Impact of the Object Constraint Language in UML-Based Development," Carleton University, Technical Report SCE-03-22, www.sce.carleton.ca/Squall, 2003.
- [5] B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering - Conquering Complex and Changing Systems*, Prentice Hall, 2000.
- [6] S. Cook, A. Kleppe, R. Mitchell, B. Rumpe, J. Warmer and A. Wills, "The Amsterdam Manifesto on OCL," in T. Clark and J. Warmer Eds., *Object Modeling with the OCL, The Rationale behind the Object Constraint Language*, Springer Verlag, pp. 115-149, 2002.
- [7] J. L. Devore and N. Farnum, *Applied Statistics for Engineers and Scientists*, Duxbury, 1999.
- [8] A. L. Edwards, *Experimental Design in Psychological Research*, Harper & Row, 1985.
- [9] H. Gomaa, *Designing Concurrent, Distributed, and Real-Time Applications with UML*, Addison Wesley, 2000.
- [10] G. Iversen and H. Norpoth, *Analysis of Variance*, Sage Publications, 1987.
- [11] A. Kleppe, J. Warmer and W. Bast, *MDA Explained - The Model Driven Architecture: Practice and Promise*, Addison-Wesley, 2003.
- [12] C. Larman, *Applying UML and Patterns - An Introduction to Object-Oriented Analysis and Design and the Unified Process*, Prentice Hall, 2002.
- [13] R. Mitchell and J. McKim, *Design by Contract, by Example*, Addison-Wesley, 2001.
- [14] A. N. Oppenheim, *Questionnaire Design, Interviewing and Attitude Measurement*, Pinter Publishers, 1992.
- [15] S. L. Pfleeger, "Understanding and improving technology transfer in software engineering," *Journal of Systems and Software*, 47 (2-3), pp. 111-124, 1999.
- [16] S. L. Pfleeger and L. Hatton, "Investigating the Influence of Formal Methods," *Computer*, vol. 30 (2), pp. 33-43, 1997.
- [17] A. E. K. Sobel and M. R. Clarkson, "Formal Methods Application: An Empirical Tale of Software Development," *IEEE TSE*, 28 (3), pp. 308-320, 2002.
- [18] J. Warmer and A. Kleppe, *The Object Constraint Language*, Addison-Wesley, 1999.
- [19] C. Wohlin, P. Runeson, M. Host, M. C. Ohlsson, B. Regnell and A. Wesslen, *Experimentation in Software Engineering - An Introduction*, Kluwer, 2000.